

Análisis y Diseño con Patrones

Práctica 1: Principios de diseño, patrones de análisis y arquitectónicos

(evaluada sobre 50 puntos)

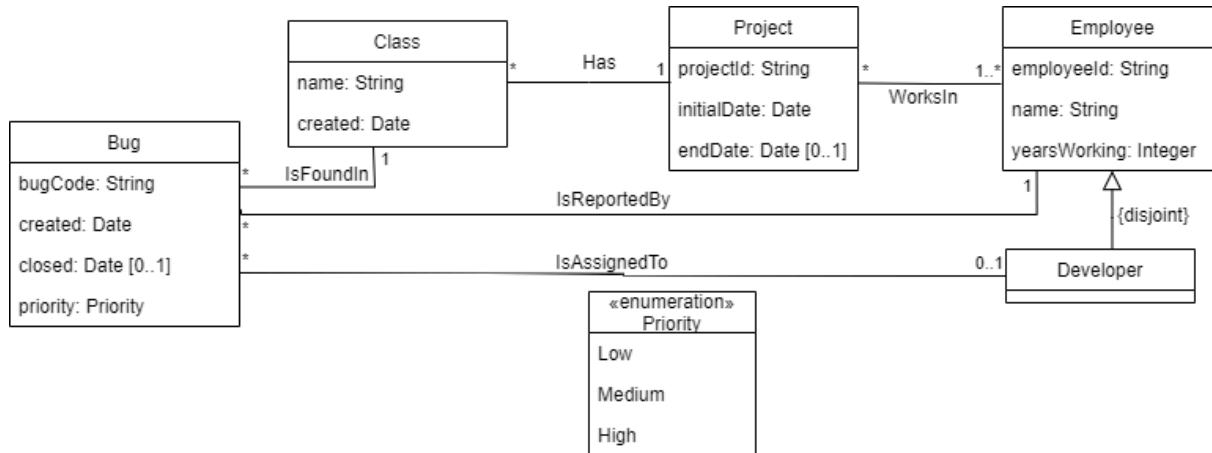
Una empresa quiere crear un software para gestionar los bugs que se producen en el desarrollo de proyectos de software orientados a objetos. Por eso, nos pide que diseñemos una parte del software para gestionar la asignación a desarrolladores, de los bugs que se detectan en los proyectos. Cada proyecto que desarrolla una empresa tiene un identificador de proyecto, una fecha de inicio, una fecha de fin (que puede no tener valor si el proyecto todavía está en curso), el conjunto de clases de objetos definidas en el proyecto y el conjunto de empleados que trabajan en el proyecto.

De las clases de objetos conocemos su nombre (que las identifica dentro del proyecto), su fecha de creación y los bugs hallados en la clase. Los empleados tienen un identificador de empleado, nombre y antigüedad. Existe un subconjunto de empleados que son los que tienen el rol de programadores.

Los bugs se identifican por el identificador del proyecto, el nombre de la clase de objetos donde se encuentra el bug y por el código del bug. Además, de los bugs sabemos la fecha de apertura, la fecha de cierre del bug (que puede no tener valor, si el bug todavía no se ha resuelto), el empleado que reportó el bug y el empleado al que se asigna la resolución del bug (que puede ser diferente al empleado que le reportó).

Disponemos ya de un análisis previo que podemos utilizar como punto de partida, pero que habrá que corregir y mejorar. A continuación disponéis del diagrama estático de análisis para esta parte del software.

Diagrama estático de análisis:



Claves de las clases:

- Project: projectId
- Class: projectId, name
- Bug: projectId, name, bugCode
- Employee: employeeId

Restricciones de integridad:

- La fecha final de un proyecto, si existe, debe ser posterior a la fecha de inicio del proyecto.
- La fecha de creación de una clase de un proyecto es posterior o igual a la fecha de inicio del proyecto y anterior a la fecha de finalización (si existe).
- La fecha de apertura de un bug debe ser posterior a la fecha de creación de la clase en la que se ha encontrado el bug.
- La fecha de cierre de un bug debe ser posterior a la fecha de su apertura.
- El bug de un proyecto debe ser reportado por un empleado del mismo proyecto.
- El bug de un proyecto debe ser asignado a un desarrollador del propio proyecto.

Pregunta 1 (8 puntos)

Enunciado

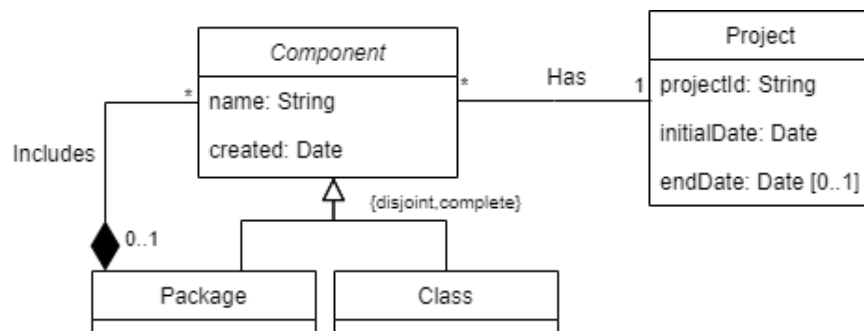
Después de realizar el análisis previo, nos damos cuenta de que además de grabar las clases de los proyectos, también queremos grabar los paquetes donde están definidas estas clases. Por tanto, los proyectos tendrán componentes que serán de dos tipos diferentes: los paquetes y las clases de objetos. Los paquetes, que tienen un nombre que los identifica y una fecha de creación, sólo pueden contener otros paquetes, paquetes y clases de objetos o sólo clases de objetos. Evidentemente, un paquete no puede contenerse a sí mismo ni directa, ni indirectamente. Asumamos que una clase de objetos o un paquete puede moverse a un paquete creado posteriormente.

Se pide:

- (2 puntos) Explica cuál de los patrones de análisis explicados en los materiales utilizarías para representar esta información. Razona la respuesta.
- (6 puntos) Propón una solución detallada en forma de diagrama estático de análisis para representar esta información (incluyendo los atributos, multiplicidades de las asociaciones, y restricciones de integridad). Muestra sólo las clases que cambien respecto al diagrama del enunciado pero incluye en el diagrama también la clase proyecto y la asociación Has.

Solución:

- a) Para representar esta información aplicamos el patrón Objeto Compuesto. Ahora, los componentes que tienen los proyectos son de dos tipos (subclases): las clases de objetos (que ya existían) y los paquetes. Los paquetes contienen otros paquetes y/o clases de objetos.
- b) A continuación disponéis del fragmento del diagrama estático de análisis que representa la información de los distintos tipos de componentes. Se han añadido también las restricciones de integridad.



Restricciones de integridad:

- (Cambio respecto a la restricción del enunciado) La fecha de creación de un componente de un proyecto es posterior o igual a la fecha de inicio del proyecto y anterior a la fecha de finalización (si existe).
- Un paquete no puede incluirse a sí mismo ni directa ni indirectamente.

Pregunta 2 (12 puntos)

Enunciado

En la versión actual del software que gestiona los bugs que se producen en el desarrollo de proyectos software se graba el desarrollador encargado de resolver un bug (asociación `IsAssignedTo`). Nos damos cuenta de que, en ocasiones, es necesario reasignar un bug a otro desarrollador ya que el asignado previamente no sabe resolverlo, e incluso, si no hay desarrolladores que se puedan hacer cargo, se puede dejar temporalmente sin desarrollador asignado. Así, queremos almacenar todos los desarrolladores que han sido asignados a los bugs y queremos saber el período en el que estuvieron asignados.

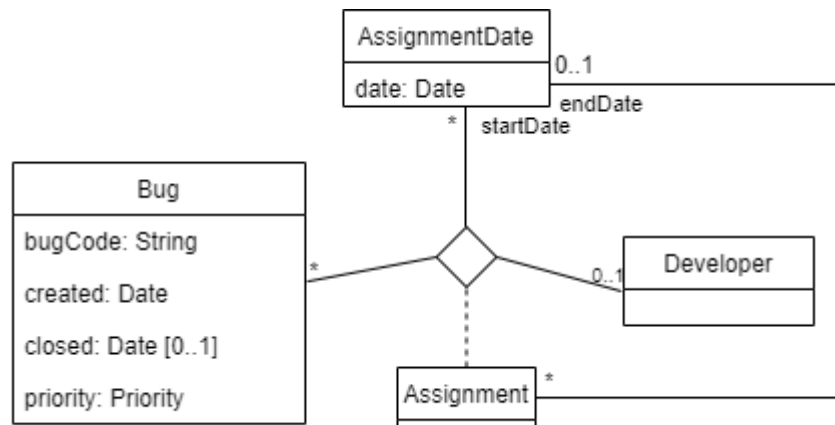
Nos proponen buscar una solución que nos permita representar esa información en nuestro sistema. En concreto, se pide:

- a) (2 puntos) Explica cuál de los patrones de análisis explicados en los materiales utilizarías para representar esta información. Razona la respuesta.
- b) (5 puntos) Propón una solución detallada en forma de diagrama estático de análisis (incluyendo los nuevos atributos que aparezcan, las restricciones de integridad y las multiplicidades de las asociaciones). Muestra sólo las clases que cambien respecto al diagrama del enunciado.
- c) (5 puntos) Propón el pseudocódigo de la operación *priorityBugs()*: *Set(Tuple(projId: String, className: String, bugCode: String))* de la clase *Developer* que devuelve el identificador de los bugs que tiene asignados para resolver y que tiene una prioridad Alta. Podéis obviar el pseudocódigo de los getters de las clases. La operación debe generar el acoplamiento más bajo posible.

Nota: Para hacer esta pregunta podéis partir del diagrama estático de análisis del enunciado.

Solución:

- a) Para poder registrar las diferentes asignaciones que tiene un bug durante el tiempo, podemos utilizar el patrón Asociación Histórica. Otras soluciones, como por ejemplo, utilizando el patrón Rango que cumplan con la semántica indicada en el enunciado también serán consideradas válidas.
- b) A continuación disponéis de la parte del diagrama estático de análisis que se ve modificado después de la aplicación del patrón Asociación Histórica.

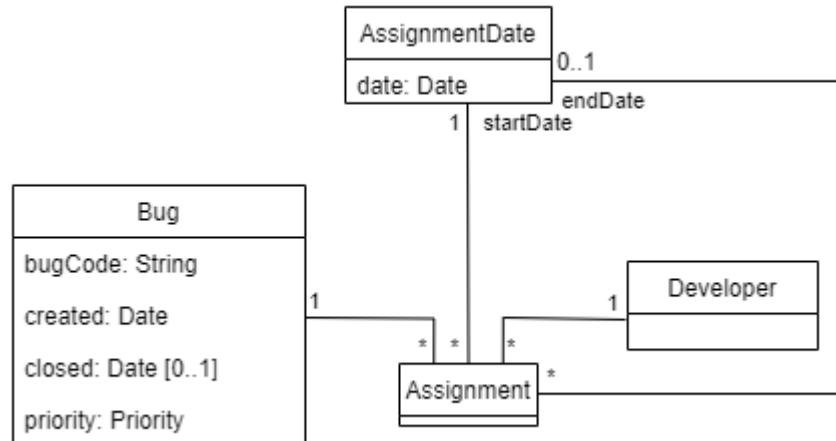


Restricciones de integridad:

- La fecha inicial de la asignación es anterior o igual a la fecha de finalización de la asignación.
- La fecha de apertura de un bug debe ser anterior o igual a la fecha de inicio más baja de todas las asignaciones que tenga el bug.
- Un bug no puede tener asignaciones solapadas.

Los lenguajes de programación orientados a objetos actuales no permiten definir asociaciones ternarias ni clases asociativas. Para poder trabajar con estos constructores, es necesario aplicar una pequeña transformación al diagrama estático de análisis para obtener

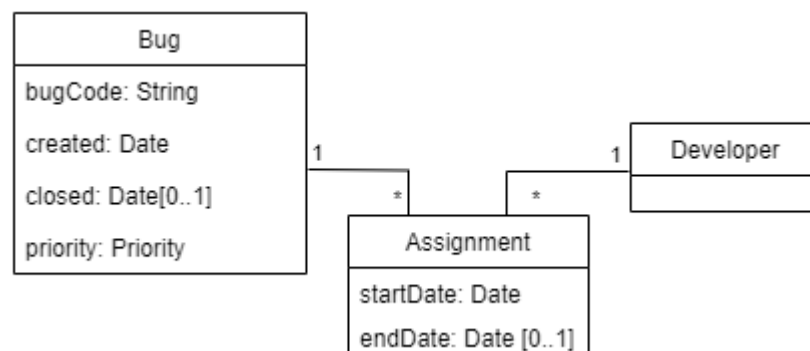
otro diagrama de clases que represente la misma semántica, pero con constructores que los lenguajes de programación orientados a objetos permitan representar. Aplicando esta transformación obtendríamos:



Restricciones de integridad (adicionales en esta solución):

- Clave de Asignación: *projectId (Project) + name (Class) + bugCode (Bug) + startDate (AssignmentDate)*

De hecho, como *AssignmentDate* podría considerarse como un tipo de datos, y los tipos de datos se pueden poner normalmente como atributos, esta solución la podemos transformar en:



Este diagrama es el que utilizaremos para realizar el apartado siguiente.

c)

```
public class Developer
{
    private List<Assignment> assignments;

    public List<BugIdentifier> priorityBugs()
    {
        List<BugIdentifier> priorityBugs = new List<BugIdentifier>();

        foreach (Assignment a in assignments) {
            if (a.active() && a.priority() == High) {
                priorityBugs.add(a.getBugIdentifier());
            }
        }
        return priorityBugs;
    }
}

public class Assignment
{
    private date startDate;
    private date endDate;
    private Developer developer;
    private Bug bug;

    public Boolean active()
    {
        Boolean act;

        if (endDate == NULL) {
            act = true;
        } else {
            act = false;
        }
        return act;
    }

    public Priority priority()
```



```
{
    return bug.getPriority();
}

public BugIdentifier getBugIdentifier()
{
    BugIdentifier bugId = new BugIdentifier();

    bugId.setProjectId(bug.getProjectId());
    bugId.setClassName(bug.getClassName());
    bugId.setBugCode(bug.getBugCode());
    return (bugId);
}

public class Bug
{
    private String bugCode;
    private date created;
    private date closed;
    private Priority priority;
    private Class class;

    public String getProjectId()
    {
        return class.getProjectId();
    }

    public String getClassName()
    {
        return class.getName();
    }

}

public class Class
{
    private String name;
    private date created;
```

```
private Project project;

public String getProjectId()
{
    return project.getProjectId();
}

}

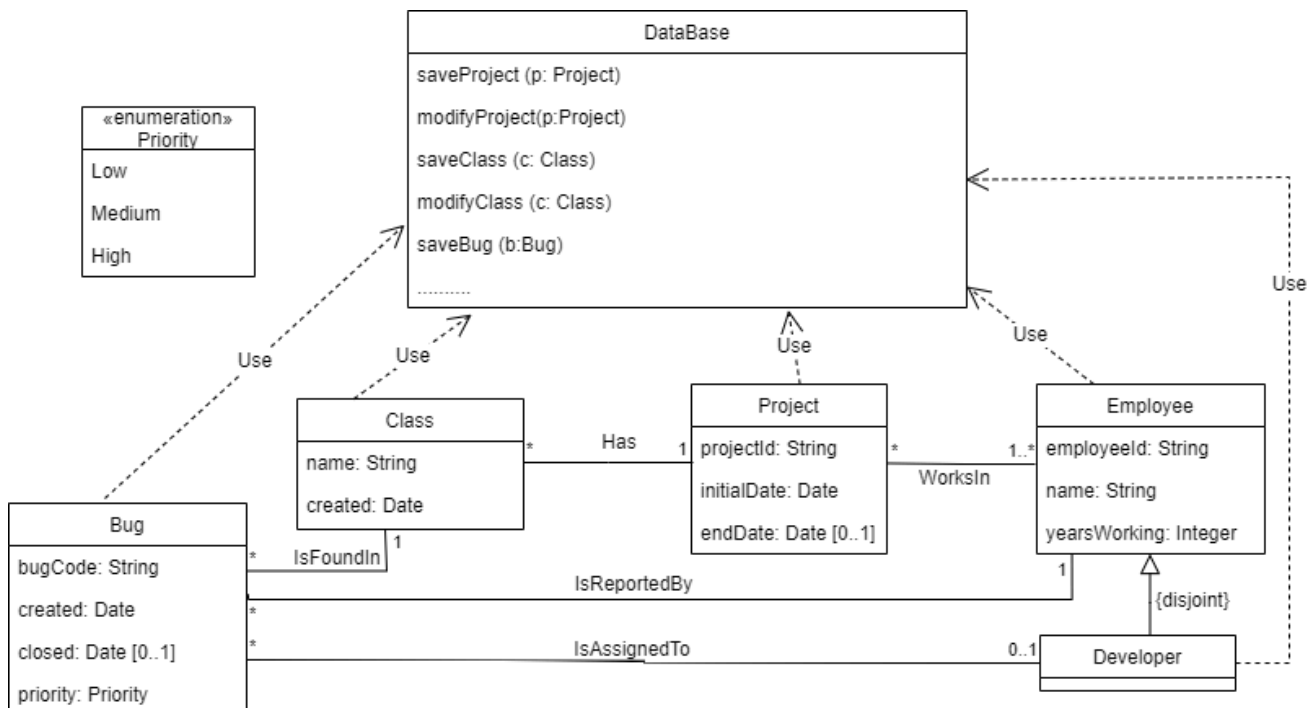
public class BugIdentifier
{
    private String projectId;
    private String className;
    private String bugCode;
}
```

Pregunta 3 (4 puntos)

Enunciado

Con el fin de hacer persistente la información de nuestro software, se decide añadir una nueva clase de objetos DataBase, con las operaciones necesarias para obtener y almacenar la información en su base de datos. La clase DataBase tiene operaciones por cada una de las clases que hay en el diagrama estático de análisis. Por ejemplo, para la clase Project, entre otras, están las operaciones: saveProject (p: Project), para almacenar en la base de datos los datos del proyecto p cuando éste no existe y modifyProject (p:Project) que modifica en la base de datos los datos del proyecto existente p. Las operaciones saveProject y modifyProject están invocadas desde operaciones de la clase Project. A continuación dispone de un fragmento del diagrama de clases con esta nueva clase.

Nota: Para hacer esta pregunta podéis partir del diagrama estático de análisis del enunciado.



Se pide:

- a) (2 puntos) Indica si se satisface el principio de segregación de interfaces y razona tu respuesta.
- b) (2 puntos) Indica si se satisface el principio de inversión de dependencias y razona tu respuesta.

Solución:

- a) No se cumple el principio de segregación de interfaces, ya que, por ejemplo, la clase Bug depende de operaciones que no necesita. Esto implica que si en algún momento se modifican estas operaciones, la clase Bug puede verse afectada.
- b) No se cumple el principio de inversión de dependencias puesto que las operaciones de las clases Bug, Project, etc... que son de la capa de dominio (en una arquitectura con una capa de presentación, dominio y de servicios técnicos), dependen de la clase DataBase que es de un nivel de abstracción más bajo. Esta clase, que es de la capa de servicios técnicos, tiene operaciones con código de bajo nivel (código de inserción y modificación de la base de datos).

Pregunta 4 (8 puntos)

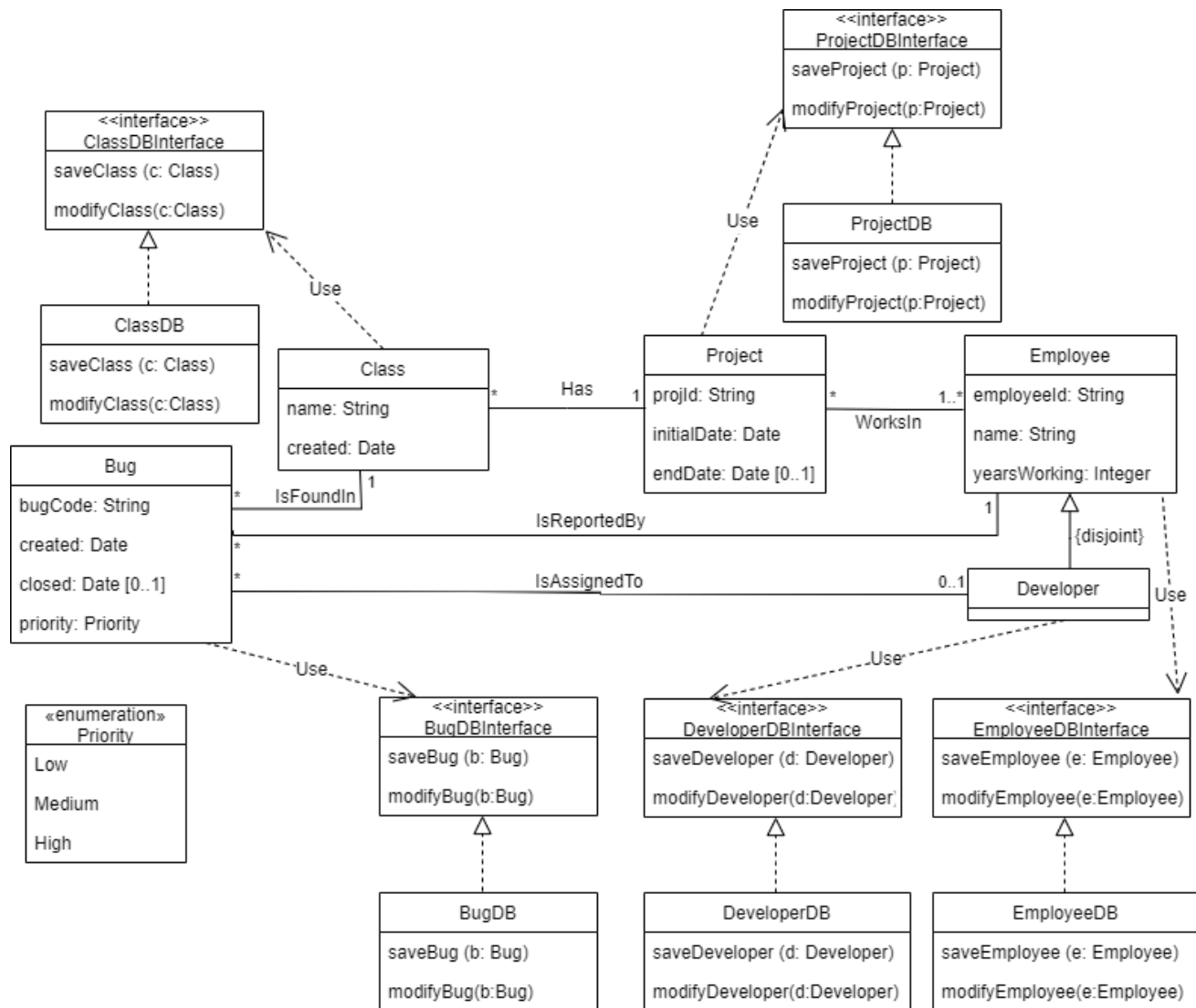
Enunciado

Si has detectado violaciones de los principios de diseño mencionados en la pregunta anterior, se pide:

- a) (4 puntos) Explica cómo solucionarías las violaciones.
- b) (4 puntos) Proporciona un diagrama de clases que satisfaga los principios de diseño violados.

Solución:

- a) Para solucionar la violación del principio de segregación de dependencias, se dividirá la clase DataBase en clases más pequeñas, de forma que cada clase sólo sea usada por la clase de dominio que la necesita. Para solucionar la violación del principio de inversión de dependencias, se añadirá una interfaz (que formará parte de la capa de dominio) entre las clases de dominio y cada clase de la capa de servicios técnicos. Estas interfaces permiten desacoplar las clases de dominio de los detalles técnicos existentes en las operaciones de la clase DataBase.
- b) A continuación se proporciona el diagrama de clases que satisface los principios de diseño violados:



Pregunta 5 (12 puntos)

Enunciado

Queremos utilizar el patrón de arquitectura Modelo, Vista y Controlador (MVC) para definir la capa de presentación del software que estamos desarrollando. Empezamos el diseño por el caso de uso "Asignar un desarrollador a un bug". Este caso de uso debe permitir al project manager de un proyecto asignar los bugs a los desarrolladores del proyecto. El caso de uso comienza cuando el project manager selecciona en un desplegable los bugs que no están asignados, el bug que desea asignar (podéis asumir que la vista ya dispone de los bugs que no están asignados para poder ser mostrados). Una vez seleccionado, el sistema muestra un desplegable con todos los desarrolladores que pueden ser asignados a ese bug. Cuando se selecciona el desarrollador, el sistema registra la asignación y devuelve el mensaje "Successfully assigned".

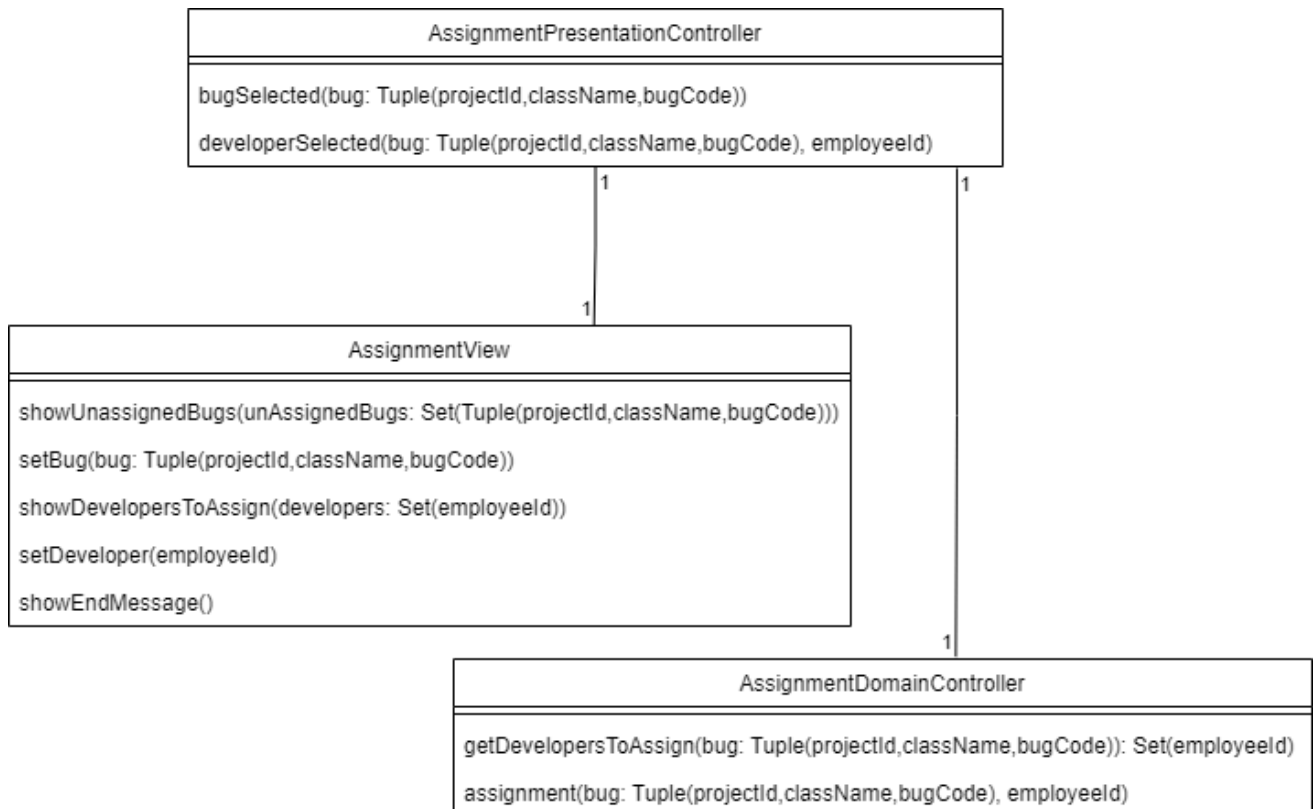
Se pide:

- a) (8 puntos) Diagrama de clases del MVC para este caso de uso incluyendo todas las operaciones que sean necesarias (las operaciones deben contener los parámetros que necesiten y los tipos de retorno, si las tienen) para desarrollar el caso de uso. Además, debéis proporcionar una pequeña descripción de cada una de las operaciones. El diagrama de clases y las operaciones deben ser específicas para este caso de uso.
- b) (4 puntos) Diagrama de secuencia o pseudocódigo del caso de uso, desde que se muestra por pantalla con todos los bugs que no están asignados, hasta que el sistema muestra el mensaje "Successfully assigned".

Nota: Para hacer esta pregunta podéis partir del diagrama estático de análisis del enunciado.

Solución:

a) Diagrama de clases y descripción de las operaciones:



A continuación se describen las operaciones de cada clase:

Operaciones de la clase AssignmentView:

- showUnassignedBugs(unAssignedBugs: Set (Tuple(projectId: String, className: String, bugCode: String))) muestra la vista con las distintas etiquetas y en un desplegable los bugs que todavía no tienen un desarrollador asignado.
- setBug(bug: Tuple (projectId: String, className: String, bugCode: String)) graba el bug a asignar. Sólo si se desea grabar el bug.
- showDevelopersToAssign(developers: Set (employeeId: String)) muestra la vista con las diferentes etiquetas y en un desplegable los desarrolladores que pueden asignarse.

- `setDeveloper(employeeId: String)` graba al desarrollador a asignar. Sólo si se desea grabar el desarrollador.
- `showEndMessage()` muestra la vista con el mensaje "Successfully assigned".

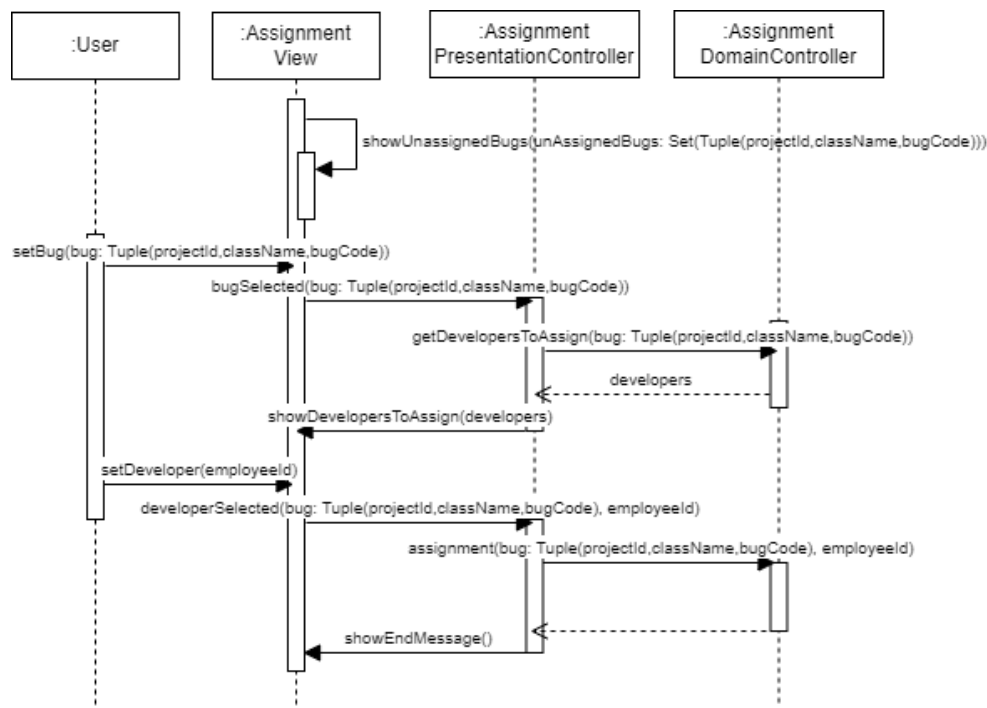
Operaciones de la clase `AssignmentPresentationController`:

- `bugSelected(bug: Tuple (projectId: String, className: String, bugCode: String))` detecta que el usuario ha seleccionado un bug a asignar.
- `developerSelected(bug: Tuple (projectId: String, className: String, bugCode: String), employeeId: String)` detecta que el usuario ha introducido el desarrollador a asignar.

Operaciones de la clase `AssignmentDomainController`:

- `getDevelopersToAssign(bug: Tuple (projectId: String, className: String, bugCode: String)): Set(employeeId:String)` obtiene los identificadores de los desarrolladores que pueden ser asignados al bug indicado en el parámetro. Nótese que los desarrolladores deben ser del mismo proyecto que el bug.
- `assignment(bug: Tuple (projectId: String, className: String, bugCode: String), employeeId:String)` asigna el bug al developer indicado en los parámetros.

b) Diagrama de secuencia:



Pregunta 6 (6 puntos)

Enunciado

Para acabar de diseñar la arquitectura en 3 capas del software que estamos desarrollando, utilizando las tres capas clásicas de muchos sistemas de información:

1. Presentación
2. Dominio
3. Servicios técnicos (Datos)

Queremos reflexionar sobre a qué capa pertenece cada una de las clases que han ido surgiendo en el enunciado (y en la solución que has elaborado hasta ahora):

- Clases del diagrama de clases del ejercicio 1.
 - Clases del diagrama de clases del ejercicio 2.
 - Clases del diagrama de clases del ejercicio 4.
 - Clases del diagrama de clases del ejercicio 5.
-
- a) (2 puntos) Indica qué clases (e interfaces, si las hay) pertenecen a la capa de servicios técnicos (en nuestro caso, capa de datos). Razona tu respuesta.
 - b) (2 puntos) Indica qué clases (e interfaces, si las hay) pertenecen a la capa de dominio. Razona tu respuesta.
 - c) (2 puntos) Indica qué clases (e interfaces, si las hay) pertenecen a la capa de presentación. Razona tu respuesta.

Solución:

- a) Las clases de la capa de datos son aquellas que gestionan el acceso a la DB. En nuestro caso, estas clases serían BugDB, ProjectDB, ClassDB, EmployeeDB, DeveloperDB.
- b) Las clases de la capa de dominio es donde está la lógica de negocio, donde trabajamos el problema que nuestro sistema resuelve. Esto incluye, naturalmente, las clases cuyas instancias representan objetos, documentos o valores del mundo real:

- Bug, Project, Class, Employee, Developer, Component, Package, Assignment.

También incluye la clase que ofrece operaciones del nivel del dominio para ser usadas por la capa de presentación, lo que llamamos controladores de dominio:

- AssignmentDomainController (de la solución del ejercicio 5)

Por último, las interfaces que representan el conjunto de objetos de dominio que el sistema conoce y que eventualmente serán implementadas por la capa de servicios técnicos para ofrecer persistencia:

- BugDBInterface, ProjectDBInterface, ClassDBInterface, EmployeeDBInterface, DeveloperDBInterface (de la solución del ejercicio 4)

- c) Las clases de la capa de presentación son las que gestionan la presentación de la información (vistas del MVC) y la recepción de los eventos en el sistema (controladores de presentación del MVC). En nuestro caso, estas clases serían AssignmentView y AssignmentPresentationController.